# Cyrinx: A Measured 36.6/27.3 kbps Bidirectional Acoustic Data Link Between a Laptop and a Phone

An empirical account of building, breaking, and verifying a wideband OFDM acoustic modem on commodity hardware

David E. Weekly*

June 9, 2026

**Abstract**

We report a software acoustic data link between a MacBook Pro (M4) speaker/microphone pair and a Google Pixel 7a resting on the laptop's palm rest, achieving measured, byte-verified over-the-air goodput of **36,571 bps** Mac→Pixel (demodulated *on the phone* by a Kotlin port of the receiver) and **27,307 bps** Pixel→Mac, against a target of 20 kbps in each direction. Goodput is counted conservatively: only payload bytes that are both CRC32-valid and, under *ordered* verification, byte-identical to the transmitted `DetRng` payload at their correct ordered position within the attributed frame (not mere set membership), divided by the full airtime span including preambles, channel-estimation symbols, pilots, FEC redundancy, CRCs, and inter-frame gaps. The modem is a 48 kHz OFDM design (FFT 2048, cyclic prefix 768) with comb pilots, per-symbol timing/phase tracking, uniform 16-QAM, and a punctured rate-3/4 $K$=7 convolutional code. Reaching the target required finding and fixing four physical-layer defects that are invisible to simulation: inter-symbol interference from a 10–22 ms delay spread, decorrelated dual-speaker transmission, a stream-end fade in the macOS output chain, and Viterbi poisoning by overconfident log-likelihood ratios from corrupted symbols. We document the discriminating experiments used to isolate each defect—including the negative results that exonerated "obvious" suspects—and contrast the delivered system with both the original design document and a predecessor stack whose theoretical 22 kbps claim corresponded to a measured 0.27 kbps. We argue that for acoustic modems on consumer hardware, the dominant engineering cost is the gap between the channel model and the channel. None of the modulation or coding machinery here is novel — OFDM, cyclic prefixes, comb pilots, QAM, convolutional/Viterbi coding, CRC block verification, and acoustic data-over-sound are all long established; the contribution is the *measured* end-to-end system on commodity heterogeneous hardware, the discriminating-experiment diagnostic methodology, the physical/platform defect catalog, and on-device verification, not new modulation theory. We frame the result accordingly: this is a well-instrumented empirical case study of how much commodity hardware delivers after real-channel debugging—validated on two device pairs (a Pixel 7a and an iPhone 17 Pro Max, each against the same MacBook Pro) in a single contact-range geometry, with same-hardware baselines against deployed tools—and a carefully measured contact-range prototype, not a broadly general or new-modulation acoustic-modem result.

*Primatech Paper Co LLC, `david@weekly.org`. The modem, diagnostics, and this whitepaper were developed with substantial AI-agent assistance (Claude Code); all reported results are from over-the-air measurements on physical hardware recorded in the project's lab notebook.

# 1 Introduction and Motivation

Consumer laptops and phones ship with speakers and microphones of considerably higher fidelity than the "data-over-sound" literature typically assumes — fidelity the acoustic-*sensing* literature already exploits aggressively [20] — yet deployed acoustic-communication systems cluster at very low rates: roughly $10^2$ bps for robust spread-spectrum schemes and roughly $10^3$ bps for multi-tone FSK [4, 7]. Acoustic links remain attractive for proximity-bound exchange—pairing tokens, small payloads, air-gapped transfer—because acoustic signals attenuate far faster and are more locality-bound than RF—especially the audible wideband link used here, which does not mean sound cannot penetrate walls, only that it does so far more weakly—require no radio pairing ritual, and use hardware already present on billions of devices [1, 10].

The Cyrinx project asked a narrower, harder question: *how much measured goodput can two specific commodity devices actually exchange acoustically?* The target was at least 20 kbps of verified goodput in each direction between a MacBook Pro (M4) and a Pixel 7a lying face-up on the laptop's palm rest (on a soft cloth, both devices at maximum volume, normal office ambient noise). The emphasis on *measured* is deliberate. An earlier in-repo protocol stack claimed "22 kbps" on the basis of a subcarrier-count capacity formula; its measured goodput was under 0.3 kbps (24-byte packets every ∼700 ms)—a factor of roughly 80 between claim and reality, and a factor of ∼135 between what that stack delivered and what the channel supports (Section 6). This paper treats that episode as data: every number reported here is from an over-the-air capture on physical hardware, and we state explicitly when a figure is an estimate rather than a measurement. We are deliberate about scope: what follows is an empirical case study of two specific device pairs in one geometry, not a general acoustic-modem result.

Contributions:

1. A characterization of the near-field laptop↔phone acoustic channel (Section 3): per-band SNR in both directions, sample-clock skew, and delay spread.
2. A wideband OFDM modem design that achieves 36.6/27.3 kbps verified goodput, about 4.5–9× the original design document's own "optimal conditions" projection (Section 4).
3. A catalog of four physical-layer defects, each fatal to the link and each invisible to simulation, with the discriminating experiments (including negative results) that isolated them (Section 5).
4. A verification methodology in which the receiving phone regenerates the expected payload locally from a portable deterministic PRNG and proves goodput on-device, so no captured samples need leave the phone (Section 4.2).

# 2 Related Work

The original Cyrinx design document [33] surveyed the deployed data-over-sound landscape; we summarize that survey here and credit the systems directly.

**libquiet / Quiet Modem Project.** Quiet [1, 2] brought soft-decision decoding (via the `libcorrect` library) to open-source acoustic networking, with GMSK and PSK modes and Reed–Solomon/convolutional error correction. Its main limitation is rigidity: the user selects a fixed profile (e.g. `ultrasonic-experimental` or `audible-7k`) at initialization, and there is no feedback loop to adapt constellation density to the channel. Cyrinx inherits Quiet's conviction that soft decisions matter — indeed, our highest-leverage single fix was a soft-decision weighting refinement (Section 5).

**minimodem.** A general-purpose FSK/RTTY software modem [5]. FSK is robust to phase corruption from echoes but spectrally inefficient, and minimodem deliberately operates as a "dumb pipe" with no integrated FEC [6], delegating integrity to the application.

**ggwave.** A widely used open-source data-over-sound library using multi-tone FSK (e.g. dividing the spectrum into 96 discrete frequencies and transmitting 6 tones at once to encode 3 bytes) [7]. Energy-detection demodulation makes it resilient to multipath, but it does not exploit high-SNR channels: where QAM can carry 4–6 bits per Hz, MT-FSK carries less than one. Rather than rely on across-paper rate comparisons, we ran ggwave and minimodem over our *own* hardware and geometry as same-channel baselines (Section 6.4, Table 6).

**Google Nearby Messages.** The (since deprecated) audio modality of Nearby used direct-sequence spread spectrum with a 127-chip code, trading nearly all throughput for detection probability: approximately 94.5 bps [4, 3]. Sufficient for a token, far from bulk transfer.

**Chirp.io and LISNR.** Commercial closed-source systems [9, 10] that emphasized the *wake-up problem*: a cheap, low-power preamble detector that gates the expensive DSP. LISNR's "Kilo Audio Bit" product claims throughputs around 1 kbps. Cyrinx adopts the preamble-gated architecture (a chirp matched filter gates the OFDM decoder) while being fully open about its error-correction internals.

**BatNet.** The most directly comparable prior art for a phone-to-phone ultrasonic link: a smartphone system that carries data between built-in speakers and microphones using 8-PSK in the 20–24 kHz inaudible band, reporting over 600 bps at up to 6 m [8]. BatNet is the established demonstration that consumer phone transducers can sustain a coherent (phase-modulated) ultrasonic link at all; our inaudible-band results (Section 8) are consistent with it on one direction and illuminate why the other direction fails on the specific Pixel 7a micro-speaker.

**Synchronization literature.** The predecessor Cyrinx stack used Zadoff–Chu CAZAC sequences for preamble synchronization and CFO estimation, following standard practice [11, 12], and its robust control-header modulation ("D-CSS") was inspired by LoRa-style chirp spread spectrum [13]. The bulk PHY described in this paper replaced the ZC preamble with a linear chirp plus known OFDM sync symbols, and replaced one-shot CFO estimation with continuous per-symbol pilot tracking—a change forced by measurement (Section 5). Pilot-based joint tracking of sampling-clock and carrier offsets is itself standard OFDM receiver practice [14, 15], and clock drift between independently clocked audio devices is a long-known problem in acoustic echo cancellation [16]; we claim no novelty here either — the finding is only that on this channel continuous tracking is mandatory rather than optional.

**Platform audio stacks.** Prior practitioners documented the hostility of consumer audio stacks to modem waveforms: voice-processing I/O units inject echo cancellation and noise suppression that attenuate steady carriers [28, 29], and macOS applies its own processing to chirps [30]. Our Android findings (Section 7) are the same lesson on a different OS: `AudioSource.UNPROCESSED` is load-bearing, and the capture path can be silently zeroed by UI state.

Academic work on aerial acoustic communication with consumer hardware [3] has demonstrated kbps-class links in the near-ultrasonic band; our contribution is less a novel modulation than a carefully verified data point—tens of kbps, bidirectional, decoded on the receiving phone—and a reproducible account of what it took to get there.

**On originality.** We want to be explicit that the core communications techniques used here are not original to this work. OFDM, cyclic prefixes, comb pilots, QAM, punctured convolutional codes with Viterbi decoding, CRC block verification, acoustic speaker-to-microphone links, and ultrasonic data-over-sound are all long established, and every system surveyed above predates and informs ours. We invented no new modulation, code, or synchronization primitive. The contribution of this paper is instead: (i) a *measured*, byte-verified end-to-end link on commodity heterogeneous hardware (a

**Table 1:** Measured over-the-air SNR by band (Welch PSD, probe-on vs. ambient floor), 2026-06-09. M→A = Mac speaker to Pixel 7a bottom mic; A→M = Pixel speaker to Mac mic.

| Band (kHz) | M→A SNR (dB) | A→M SNR (dB) |
|---|---|---|
| 0.3–2 | 26.4 | 28.1 |
| 2–6 | 41.2 | 44.3 |
| 6–10 | 37.1 | 47.3 |
| 10–14 | 36.0 | 45.5 |
| 14–18 | 38.1 | 33.9 |
| 18–21 | 36.1 | 10.0 |
| 21–24 | 40.7 | 4.6 |

MacBook Pro and a Pixel 7a, decoded on the phone); (ii) a discriminating-experiment diagnostic methodology, reported with its negative results; (iii) a catalog of physical-layer and platform defects that are invisible to simulation; and (iv) on-device verification via a portable deterministic PRBS. The novelty is empirical and methodological, not theoretical.

# 3 Channel Characterization

## 3.1 Methodology

All characterization was over-the-air at the final physical geometry (Pixel 7a face-up on the MacBook Pro palm rest on a soft cloth). Three probe types were used, none requiring synchronization between the devices:

- **Probe-on vs. probe-off PSD.** A random-phase multitone spanning 0.3–23.5 kHz at digital amplitude 0.6 was played while the far device recorded; Welch periodograms (NFFT 1024, 46.875 Hz bins) of probe-on and 6 s of ambient floor yield per-band SNR without any time alignment.
- **Exponential sine sweep (ESS)** for impulse response and delay spread.
- **Long pure tone** (10 kHz, 8 s) for sample-clock offset, via a quadratically interpolated FFT peak.

## 3.2 Measurements

Table 1 gives the measured SNR by band in both directions (Mac→Android, denoted M→A, uses the Pixel's bottom microphone, channel 0).

Key findings, several of which contradicted prior assumptions:

- **The Mac→Pixel channel is essentially flat to 23 kHz** in this near-field geometry. An earlier claim in the project history of "−32.5 dB roll-off at 16 kHz" did not reproduce; design decisions based on it (a narrow 10–14 kHz "flat shelf" band) were wrong.
- **The Pixel→Mac direction dies above ∼17 kHz**: 10.0 dB SNR at 18–21 kHz falling to 4.6 dB at 21–24 kHz (Pixel speaker and/or Mac microphone roll-off). This asymmetry has direct consequences for any inaudible-band variant (Section 8).

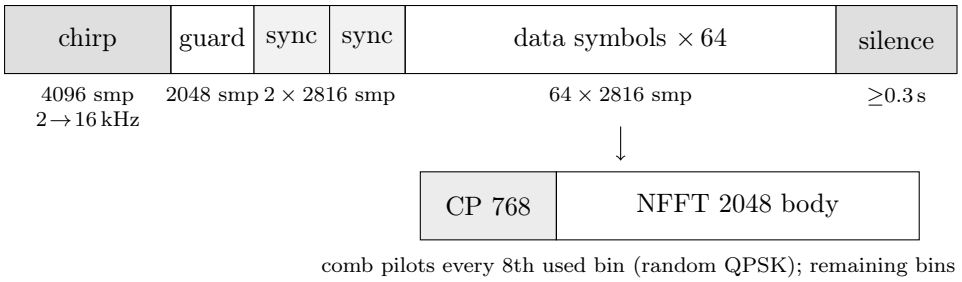


**Figure 1:** Bulk-PHY frame structure (widths not to scale). The chirp gates the decoder and provides coarse timing; the guard lets the chirp's reverberant tail decay; two known QPSK sync symbols give the least-squares channel estimate and a per-bin noise estimate; 64 data symbols follow. The trailing in-stream silence defends the final symbol against the macOS stream-end fade (Section 5). One frame spans 192,000 samples = 4.0 s.

- **Sample-clock offset between the two devices is −24.6 ppm**, which at the final OFDM geometry corresponds to about 0.037 samples of drift per symbol — small per symbol, but ruinous over a frame without continuous tracking.
- **Delay spread (ESS, −30 dB threshold) is ∼21.7 ms M→A and ∼10.8 ms A→M.** Most energy arrives in the first few ms, but the reverberant tail is long; this is roughly an order of magnitude larger than the 2 ms the original design document budgeted for.
- **The Pixel's bottom microphone beats its top microphone by 6–20 dB below 10 kHz** at this geometry.

From Table 1, Shannon capacity with an 8-bit-per-symbol cap is approximately 184 kbps M→A and 155 kbps A→M (an estimate, not a measurement). The achieved link uses roughly 20–24% of that, so substantial headroom remains.

# 4 System Design

## 4.1 Modem architecture

The link is a half-duplex, open-loop bulk-transfer PHY at 48 kHz PCM16. The OFDM geometry is NFFT 2048 (23.4 Hz bins) with a 768-sample cyclic prefix (16 ms), giving 17.05 symbols/s. To be precise about why this works: the 16 ms CP does *not* cover the entire measured Mac→Pixel delay spread of ∼21.7 ms (at the −30 dB threshold). The link succeeds because most of the channel energy arrives early—well within the CP—and the receiver tolerates the residual low-level reverberant tail beyond it, not because the CP spans the full measured tail. The CP length was chosen against the measured delay spread, not from a model, but it is an energy-coverage choice, not a worst-case-tail guarantee. Direction-specific band profiles follow the measured channel: 1.1–23 kHz Mac→Pixel and 0.6–17 kHz Pixel→Mac (Table 2).

Each frame (Figure 1) consists of:

1. A 4096-sample linear chirp (2→16 kHz) for detection and coarse synchronization, found by matched filter with threshold-and-suppression peak picking for multi-frame runs.
2. 2048 samples of guard silence so the chirp's reverberation decays before channel estimation.

**Table 2:** Direction-specific transmit profiles (16-QAM, rate 3/4, 64 data symbols/frame). Bin counts and per-frame payload follow from the band edges and the 23.4 Hz bin spacing.

| | Mac→Pixel | Pixel→Mac |
|---|---|---|
| Band | 1.1–23 kHz | 0.6–17 kHz |
| Used bins | 935 | 700 |
| of which pilots | 117 | 88 |
| of which data (16-QAM) | 818 | 612 |
| Coded bits/symbol | 3272 | 2448 |
| CRC32 blocks/frame | 75 | 56 |
| Payload/frame | 19,200 B | 14,336 B |
| Frame airtime | 4.0 s | 4.0 s |
| Transmit quirk | left speaker only | — |

3. Two known full-band QPSK *sync symbols*. Their average gives a per-bin least-squares channel estimate $H$; their difference gives a per-bin noise-variance estimate. $H$ is deliberately *not* smoothed across bins—under multipath its phase rotates through multiple cycles across a few bins and smoothing destroys it (only the real, positive noise variance is smoothed).

4. 64 data symbols. Every 8th used bin carries a comb pilot (random QPSK); the rest carry uniform 16-QAM. Per symbol, a three-pass iterative fit of the pilot phase ramp extracts a timing slope plus common phase error and corrects all bins, absorbing the $-24.6$ ppm clock skew continuously. (Per-bin adaptive loading is implemented but was not needed to reach the target.)

FEC is a $K$=7 (171,133) convolutional code, punctured to rate 3/4 (pattern 110110), zero-terminated, with soft max-log LLRs, a frame-wide pseudorandom interleaver, and Viterbi decoding. Payload is segmented into 256-byte blocks, each with a CRC32, purely for honest goodput accounting. Two further measured-channel adaptations are part of the transmit contract: the FFT window is biased 24 samples early so multipath pre-cursors stay inside the CP, and Mac→Pixel transmission uses the *left speaker only* (Section 5).

The load-bearing receiver refinement is **per-symbol LLR weighting**: each data bin's noise variance is taken as the sync-derived per-bin estimate *plus that symbol's own pilot error-vector magnitude squared*. A symbol corrupted by a burst, dropout, or fade therefore reports honestly weak LLRs and degrades into soft erasures rather than poisoning the Viterbi traceback through the interleaver (Section 5, defect 4); converting burst corruption into erasures is the information-theoretically favorable move on a burst channel [17]. This also provides burst-noise immunity (typing, key clicks) at no added cost.

### 4.2 Portable deterministic streams and on-device verification

All deterministic streams in the modem—pilot phases, sync symbols, the interleaver permutation, payload-padding bits, and the test payload itself— derive from a shared `splitmix64` generator (`DetRng`), implemented identically in Python (`scratch/hw20k/modem.py`) and Kotlin (`BulkDemod.kt`). This small trick has an outsized methodological payoff: the phone regenerates the *expected* payload bytes locally from the seed and compares them against what it decoded, so end-to-end goodput is proven on the receiving device itself and no captured audio or decoded data needs to leave the phone. The Kotlin receiver is a port of the Python receive path (including a

double-precision radix-2 FFT written for the purpose) and was validated bit-compatible against the Python decoder on an identical capture file (225/225 blocks, matching EVM) before the live runs.

### 4.3 What the bulk PHY deliberately omits

Relative to the original design document [33], the bulk PHY drops the per-frame TDD ping-pong MAC with ACK/channel-state reports, rateless (Raptor/LDPC) coding, and the rate-adaptation "gear" state machine. The predecessor stack implemented that chatty MAC, and its turnaround gaps, tiny payloads, and ACK timeouts consumed more than 97% of airtime—this is why it measured 0.27 kbps. For bulk transfer, long streaming frames with open-loop MCS selection from the measured channel are the right shape; closed-loop adaptation belongs at session granularity, not frame granularity. Encryption (an X25519/CTR/HMAC envelope exists in the predecessor stack), discovery, and ARQ are likewise out of scope for the PHY measurement reported here.

## 5 What Didn't Work: Four Defects and the Experiments That Found Them

The first end-to-end attempts decoded zero blocks despite a channel with 35–45 dB of mid-band SNR. Four distinct physical-layer defects were stacked on top of each other; each was fatal alone, and none is visible in a simulation. All were diagnosed on real captures. We describe them in discovery order, then the diagnostic method.

### 5.1 The defects

**Defect 1: ISI — delay spread far exceeds the cyclic prefix.** The initial geometry (NFFT 1024, CP 256 = 5.3 ms) assumed a short channel. A repeated-symbol probe showed adjacent *identical* symbols agreeing at 24–27 dB while adjacent *different-content* symbols agreed at only 11–13 dB — the signature of inter-symbol interference, not noise or channel instability. The geometry moved to NFFT 2048 / CP 768 (16 ms), consistent with the measured 10–22 ms delay spread.

**Defect 2: dual-speaker transmission corrupts the composite.** With the phone on the left palm rest, the right speaker's signal arrives about 3× weaker with decorrelated phase (EVM 1.33 received alone); summed with the left speaker it drags composite EVM to ∼0.5. Transmitting from the left speaker only gives EVM ∼0.06. (A true 2×2 MIMO receiver with per-stream equalization could exploit the second speaker; it was not needed to reach the target.)

**Defect 3: stream-end fade kills the final symbol.** The macOS speaker chain tapers the last ∼10 ms when an output stream stops. Per-symbol raw BER against the known coded stream was 0.000 for symbols 0–14 and 0.264 for the final symbol. With a frame-wide interleaver, that one dead symbol contaminates *every* FEC block. Fix: at least 0.3 s of trailing silence inside the stream, so the fade lands on padding.

**Defect 4: overconfident LLRs from a corrupted symbol poison Viterbi.** Even one corrupted symbol in 16 (≈1.7% average raw BER, trivially correctable in principle) yielded only 0–1 of 6 blocks, because the corrupted symbol's wrong LLRs carried full confidence through the interleaver into the Viterbi metric. Weighting each symbol's LLRs by its own pilot $\text{EVM}^2$ turns such symbols into soft erasures; the same captured regression then decodes 6/6. This was the single highest-leverage coding change in the project.

### 5.2 Diagnostic methodology, including negative results

The defects above were separated by discriminating experiments, several of which mattered precisely because they *exonerated* a suspect. We list them as first-class results; all are reusable scripts in `scratch/hw20k/`.

- **Repeated-symbol probe.** Transmit $N$ identical OFDM symbols. Identical-vs-different-content consistency separates ISI from channel instability, dropped samples, and clock drift; the phase trace across repeats measured drift cleanly (linear in time, proportional to frequency $\Rightarrow$ pure clock skew, channel stable).
- **Amplitude ladder** (digital amplitude 0.07→0.7): EVM was flat across 20 dB of drive level, *exonerating* speaker nonlinearity and speaker-protection DSP as the dominant impairment — despite being the "obvious" suspect at maximum volume.
- **`afplay` vs. `sounddevice` A/B** on the identical frame file: identical EVM, *exonerating* the playback software path.
- **Genie TX/RX ratio test.** FFT the saved transmit waveform and the capture at matching symbol positions and compare directly: adjacent-symbol consistency held at 15 dB while sync-to-late-symbol agreement degraded smoothly — the channel was fine; the receiver's tracking was at fault.
- **Piecewise TX↔RX alignment.** Correlating successive transmit chunks against the capture detects dropped samples (none found; a smooth +24 ppm drift instead) and revealed the correlator tap-hopping between two multipath arrivals ∼19 samples apart — motivating the 24-sample-early window bias.
- **Per-symbol raw BER** against the known coded stream localized failure to exactly the final symbol (defect 3) when aggregate metrics (EVM, mean BER) merely looked "mediocre."

**A cautionary tale about debug tooling.** An early debug script mixed module-level FFT/CP constants with a differently configured modem object and fabricated a "0 dB sync consistency" reading that misdirected the investigation for a full round. The lesson is procedural: diagnostic tooling must share the *exact* configuration object with the system under test. The script survives in the repo, fixed and carrying a warning comment, as a reminder.

## 6 Evaluation

### 6.1 Stepped modulation results

With all four defects fixed, modulation and code rate were stepped upward on real over-the-air captures (offline decode; 5-frame runs; goodput includes all overhead and 0.25 s inter-frame gaps). Table 3 shows clean decodes at every step; both directions cross the 20 kbps target at 16-QAM rate 3/4.

### 6.2 Final verified measurement

The definitive run (`scratch/hw20k/final_measurement.py`) transmits 5 frames per direction at 16-QAM rate 3/4 and counts goodput as payload bytes that are both CRC32-valid *and* pass *ordered* verification: each CRC-valid 256-byte block must be byte-identical to the transmitted `DetRng` payload

**Table 3:** Stepped over-the-air results (offline decode of real captures, 2026-06-09). All blocks CRC-valid at every step.

| Direction | Profile | Blocks | EVM | Goodput |
|---|---|---|---|---|
| M→A (1.1–23 kHz) | QPSK r1/2 | 75/75 | 0.06 | 12.29 kbps |
| M→A | 16-QAM r1/2 | 150/150 | 0.06 | 24.58 kbps |
| M→A | 16-QAM r3/4 | 225/225 | 0.07 | **36.86 kbps** |
| A→M (0.6–17 kHz) | QPSK r1/2 | 54/54 | 0.12 | 8.85 kbps |
| A→M | 16-QAM r1/2 | 111/111 | 0.12 | 18.19 kbps |
| A→M | 16-QAM r3/4 | 168/168 | 0.11 | **27.53 kbps** |

**Table 4:** Final byte-verified over-the-air goodput (2026-06-09). The Mac→Pixel capture was demodulated and verified on the Pixel itself; it never left the phone.

| Direction | Decoder | Blocks verified | Goodput |
|---|---|---|---|
| Mac→Pixel 7a (1.1–23 kHz) | on the Pixel (`BulkDemod.kt`) | 375/375 (96,000 B) | **36,571 bps** |
| Pixel 7a→Mac (0.6–17 kHz) | on the Mac (`modem.py`) | 280/280 (71,680 B) | **27,307 bps** |

at its correct ordered position within the attributed frame, not merely a member of the expected block set. Goodput is those verified bytes divided by the span from the first frame's chirp to the last frame's final data sample — preambles, sync symbols, pilots, FEC redundancy, CRCs, and inter-frame gaps all count against the number. Each direction spans 21.0 s of airtime.

The ≥0.3 s of trailing silence that defends the final symbol against the macOS stream-end fade (Section 5) is part of the transmit contract but is *not* counted in the goodput span, which runs from the first chirp to the last data sample. For a long stream this silence amortizes to nothing; for the 5-frame measurement here it modestly inflates the headline ("span") goodput relative to a gross accounting that includes the trailing silence. We report the span goodput as the headline number and note that the gross figure is slightly lower (by the ratio of trailing silence to total airtime).

Both directions exceed the 20 kbps goal with zero block errors (Table 4). The on-device Kotlin decoder runs in about 170 ms per 4 s frame on the Pixel 7a — roughly 20× real time — in pure Kotlin with a double-precision radix-2 FFT — before any use of the NEON-vectorized DSP libraries available on this hardware [26, 27] — which indicates that a real-time streaming receiver is computationally comfortable on this class of phone.

### 6.3 Second device pair: iPhone 17 Pro Max

To address the single-device-pair limitation, we repeated the campaign on a second mobile device—an iPhone 17 Pro Max (`iPhone18,2`)—against the same MacBook Pro, same geometry (phone on a cloth on the palm rest), same modem (NFFT 2048, CP 768, 48 kHz, 16-QAM rate 3/4), with the same ordered verification, run as a reliability campaign of 8 runs per direction at 5 frames/run (Table 5). The iOS device is driven over USB with `xcrun devicectl` (launch with an environment-variable command, then a container file pull); the harness details are in `docs/IOS_HIL.md`.

**Mac→iPhone is essentially identical to Mac→Pixel** at $36.47 \pm 0.12$ kbps (vs. the Pixel's 36.57 kbps), decoded *on the iPhone* by a Swift port of the receiver that was validated bit-exact against the Python decoder on an identical capture (75/75 blocks at EVM 0.0223). The Mac→iPhone band is the full 1.1–23 kHz, as for the Pixel.

**Table 5:** Second-pair reliability campaign, iPhone 17 Pro Max against the same MacBook Pro, 16-QAM rate 3/4, ordered-verified (measured, 2026-06-10). 8 runs per direction, 5 frames/run. Block-failure CIs are Wilson 95%.

| | Mac→iPhone | iPhone→Mac |
|---|---|---|
| Band | 1.1–23 kHz | 0.6–11 kHz |
| Decoded on | iPhone (Swift port) | Mac (`modem.py`) |
| Mean goodput | $36.47 \pm 0.12$ kbps | $16.38 \pm 0.75$ kbps |
| Min / median / max (kbps) | 36.28 / 36.52 / 36.57 | 15.02 / 16.73 / 17.07 |
| Blocks verified | 2992/3000 | 1344/1400 |
| Block-failure rate | 0.27% (CI 0.14–0.53%) | 4.0% (CI 3.1–5.2%) |

**iPhone→Mac is lower, at $16.38 \pm 0.75$ kbps, purely because the usable band is narrower.** The iPhone speaker rolls off about 18 dB by 10–14 kHz and about 33 dB by 14–17 kHz, so the Pixel's 0.6–17 kHz return profile collapses the link; narrowing the iPhone return band to 0.6–11 kHz recovers a clean link at the lower rate. This is a transducer limit, stated as such—not a protocol or tuning failure. Across both pairs the downlink (Mac→phone) is robust and rate-matched; the uplink (phone→Mac) is bounded by the specific mobile speaker.

### 6.4 Same-hardware baselines

Comparing acoustic links across papers is treacherous because hardware, range, band, and the goodput definition all differ. To make the comparison fair we ran the deployed open-source tools *over the identical Mac→iPhone OTA path and geometry*, with the same goodput definition (correctly received payload bytes divided by message airtime). Table 6 reports these same-hardware measurements alongside two literature points, kept clearly distinguished.

The single-carrier FSK failures at 600 baud and above are an honest illustration of *why* wideband-plus-FEC matters on this channel, not a knock on minimodem: it targets a clean, narrowband channel and deliberately omits integrated FEC [6], so it has no defense against the desktop multipath that the OFDM equalizer and convolutional code absorb. At 36,470 bps verified, Cyrinx is roughly 137× the best recovered deployed-tool baseline (ggwave "Fastest" at 267 bps) on the same hardware and geometry.

### 6.5 Contrast with the predecessor stack

The predecessor in-repo stack (Zadoff–Chu sync, D-CSS headers, OFDM-QPSK payload, per-frame ACK MAC) reported a "symmetrical 20+ kbps" result that was, on inspection, a capacity formula ($86 \text{ carriers} \times 6 \text{ bits} \times 42.857 \text{ symbols/s} = 22.11$ kbps); its measured goodput was under 0.3 kbps. The project history now carries a correction notice. The channel supported roughly 135× what that stack delivered; the loss was architectural (turnaround gaps, 24-byte payloads, ACK timeouts) rather than physical. We include this not to disparage the earlier effort — its Kotlin FFT, HIL tooling, and platform groundwork carried forward — but because the simulation-vs-measurement gap is the central finding of the project.

**Table 6:** Goodput baselines. Rows marked *measured* were run by us over the *identical* Mac→iPhone 17 Pro Max OTA path and geometry, same goodput definition (recovered payload bytes / message airtime), 2026-06-10. The two *literature* rows are reported by their authors on different hardware and are not same-hardware comparisons.

| System | Band | Range | Modulation | Goodput, payload/airtime | Notes |
|---|---|---|---|---|---|
| *Measured, same Mac→iPhone channel:* | | | | | |
| ggwave "Fast" | audible | contact | MT-FSK | 133 bps | recovered |
| ggwave "Fastest" | audible | contact | MT-FSK | 267 bps | recovered; best deployed tool |
| ggwave "U-Fastest" | ultrasonic | contact | MT-FSK | 267 bps | recovered |
| minimodem 300 bd | audible | contact | FSK | 237 bps | recovered |
| minimodem 600/1200/2400 bd | audible | contact | FSK | FAIL | 1–2 garbage bytes; no FEC/eq. in desktop multipath |
| **Cyrinx (this work)** | 1.1–23 kHz | contact | 16-QAM r3/4 OFDM + conv. FEC | **36,470 bps** | verified; ∼137× best deployed tool |
| *Literature (different hardware; not same-channel):* | | | | | |
| Quiet | ultrasonic profile | — | OFDM | (profile-dependent) | [1, 2] |
| BatNet | 20–24 kHz | to 6 m | 8-PSK | >600 bps | [8] |

# 7 Platform Findings

Two classes of platform behavior materially affected measurement integrity and are worth recording for other practitioners.

**Android silently records zeros when the activity is not top-visible.** A secure-lockscreen bouncer (`AlternateBouncerView`) or an expanded notification shade both trigger the audio policy's capture silencing (`dumpsys audio` shows the record client as `silenced`); the capture API succeeds and returns buffers of zeros. Mitigations: `setShowWhenLocked(true)` and `setTurnScreenOn(true)` on the activity, collapsing the status bar (`cmd statusbar collapse`), and a wake sequence before every capture. Battery saver at low charge re-dozes the screen despite `svc power stayon`. Separately, `AudioRecord` with `AudioSource.UNPROCESSED` at 48 kHz stereo works on the Pixel 7a and is mandatory: processed sources apply AGC and noise suppression that would corrupt QAM phase.

**macOS gotchas.** The Mac microphone clips near-field at full input volume (captures rail at ±1.0); input volume around 22/100 is appropriate for a phone speaker at maximum on the palm rest. The native 48 kHz devices must be selected explicitly — external displays and Bluetooth headsets silently become the default device and hijack playback. And, as defect 3 showed, the output chain's stream-end fade is part of the channel.

# 8 Limitations and Future Directions

## 8.1 Threats to validity

We state these honestly as scope statements, so the numbers are not over-read.

1. **Device pairs.** Two device pairs are now measured—a Pixel 7a and an iPhone 17 Pro Max, each against the same MacBook Pro (Section 6.3)—which strengthens the generality of the downlink result and the uplink transducer limit. But both phones are mobile devices and the laptop side is a single Mac; a wider device matrix (other laptops, other phone classes) is untested.

2. **Single physical geometry.** Every measurement uses one geometry: the phone resting face-up on a soft cloth on the MacBook palm rest. A distance and orientation robustness sweep is identified as future work and has not yet been performed.

3. **Single ambient condition.** All runs are in one quiet room, measured near-silent (in-band RMS $\sim 5 \times 10^{-8}$). Behavior under office, street, or adversarial near-ultrasonic conditions is untested.

4. **Trailing-silence accounting.** The headline goodput is a span figure (first chirp to last data sample) that excludes the $\geq 0.3$ s trailing silence; gross goodput is slightly lower, and the gap amortizes away only for long streams (Section 6).

5. **Batch decode, not real-time streaming.** The receiver records then decodes offline. Decode runs at roughly 20× real time, so a streaming port is plausible, but a live streaming receiver is not yet implemented.

6. **Ordered verification is implemented** (Section 6): each CRC-valid block must be byte-identical to the transmitted payload at its correct ordered position, not merely a member of the expected set, so the goodput figures reflect ordered byte-exact delivery.

**Limitations of the result as it stands.** These are scope statements, not failures; we state them plainly so the result is not over-read. The measured bulk PHY: (i) is *not* integrated into the project's public Swift/C transport API — it lives in the research harness (`scratch/hw20k/`) and the Kotlin HIL decoder; (ii) is *not* real-time streaming — it is a batch design that records, then decodes offline, though the on-device decoder runs at roughly 20× real time and a streaming port is therefore plausible; (iii) is *not* closed-loop rate-adaptive — the MCS is chosen open-loop from a same-day channel measurement, with no per-frame feedback; (iv) is *not* secured by the project's existing X25519/CTR/HMAC envelope — the PHY measurement carries raw test payloads; and (v) is validated in exactly *one* physical geometry (a phone face-up on a MacBook Pro palm rest), on two device pairs (Pixel 7a and iPhone 17 Pro Max, each against the same Mac), under one ambient condition. The link is half-duplex and has no link layer. The numbers should be read as "what these hardware pairs can do at contact range," not as a general-environment claim. A product link would also have to confront the security properties of short-range audio channels, which have a substantial literature of their own — from channel-security surveys [21] to classic acoustic side channels such as keyboard emanations [22].

**Untapped physical headroom** (estimates, from the measured SNR):

- *Per-bin adaptive bit loading.* Mid-band SNR is 35–45 dB; 64/256-QAM there should roughly double throughput. The loading machinery already exists in the modem.

- *Maximal-ratio combining* of the Pixel's two microphones (the bottom mic leads the top by 6–20 dB below 10 kHz; MRC adds diversity above).
- *True 2×2 MIMO* (Mac stereo speakers × Pixel stereo mics) for spatial multiplexing Mac→Pixel — turning defect 2's decorrelation from a bug into capacity.
- *Real-time streaming RX.* At 20× real time for the batch decoder, a ring-buffer port is straightforward; real-time audio processing pipelines on Android are well documented [25].
- *Link layer:* session-granularity rate adaptation from receiver-fed-back per-bin SNR, plus ARQ for residual block errors at higher MCS.
- *Mobility.* Everything here assumes the static contact-range channel. For devices in relative motion the channel becomes doubly dispersive, and the delay–Doppler waveforms (OTFS) studied for underwater acoustic channels [18, 19] are the natural candidates — untested here.

### 8.2 Measured result: the inaudible-ultrasonic variant

The original goal (and the obvious product form) is an inaudible link in the 18.5–23.9 kHz band. We subsequently ran the bulk-PHY methodology entirely in that band at 96 kHz sampling [34]; the answer is that it is *feasible in one direction but transducer-bound in the other*.

**Mac→Pixel works.** In the 18.5–23.9 kHz band at 96 kHz (NFFT 2048, CP 768, the chirp preamble moved up into 18.6–23.4 kHz), 16-QAM rate 3/4 delivers **∼9 kbps** of verified goodput at EVM ∼0.11, with every block recovered. The ceiling here is bandwidth, not modulation: a ∼5.4 kHz coherent band at ∼22 dB SNR caps near 9–10 kbps, and reaching 20 kbps inaudibly would require either more coherent ultrasonic bandwidth than this Pixel offers or dropping below 18.5 kHz (audible).

**Pixel→Mac does not close** — and the reason is a clean physical result rather than a tuning failure. The mobile micro-speaker is *phase-incoherent* above ∼18 kHz, and we now establish this with repeated-run statistics on *two* independent phones rather than single shots. A single-tone STFT phase-jitter measurement on the iPhone 17 Pro Max speaker, 8 repetitions per condition, gives a median detrended phase standard deviation (with [min–max]) of 0.007 rad [0.006–0.013] at 8 kHz, amplitude 0.9 (coherent), versus 12.8 rad [7.5–14.0] at 19.5 kHz amp 0.9, 11.1 rad [6.4–14.4] at 19.5 kHz amp 0.4, 15.9 rad [9.5–24.2] at 22 kHz amp 0.9, and 18.9 rad [11.2–26.9] at 22 kHz amp 0.4 (Table 7). In-band received RMS at the ultrasonic tones is ∼0.005, roughly 10× weaker than 8 kHz's 0.053 — power is present, but the phase is scrambled. The Pixel 7a showed the same pattern (0.055 rad at 8 kHz; ∼10 rad at 19.5 kHz; ∼25–31 rad at 22 kHz). Two independent phones exhibiting the same collapse make this a general consumer-micro-speaker limit above ∼18 kHz, not a single-device quirk. The speaker radiates ultrasonic *power*, but cannot reproduce it with the phase coherence that OFDM/QAM — or any phase modulation — requires.

This resolves an apparent contradiction in the channel survey: a stationary multitone PSD reports ∼27.8 dB "SNR" for Pixel→Mac at 18.5–21 kHz, yet coherent OFDM demodulation in the same band yields EVM ∼1.0 (SINR ∼0 dB). Power-per-bin and phase coherence are different quantities; PSD-based channel surveys overstate capacity for phase-modulated waveforms on this transducer. A room-tone capture confirmed the band is near-silent (in-band RMS $\sim 5\times10^{-8}$), so the limit is the transmit transducer, not ambient noise. (One deployment caveat: our quiet room is not the general case — deliberate near-ultrasonic emitters such as anti-eavesdropping jammers occupy exactly this band [23, 24].)

The conclusion is that a *symmetric* inaudible link is bounded by the mobile speaker, not by the protocol. The fast inaudible direction (Mac→Pixel, ∼9 kbps) is real; the inaudible return path

**Table 7:** Single-tone STFT phase-jitter (detrended phase standard deviation), iPhone 17 Pro Max speaker, 8 reps per condition: median [min–max], rad (measured, 2026-06-10). Below ∼18 kHz the speaker is phase-coherent; above it the phase is scrambled regardless of drive amplitude. The Pixel 7a showed the same pattern.

| Tone | Amplitude | Phase-jitter std (rad) | Verdict |
|---|---|---|---|
| 8 kHz | 0.9 | 0.007 [0.006–0.013] | coherent |
| 19.5 kHz | 0.9 | 12.8 [7.5–14.0] | incoherent |
| 19.5 kHz | 0.4 | 11.1 [6.4–14.4] | incoherent |
| 22 kHz | 0.9 | 15.9 [9.5–24.2] | incoherent |
| 22 kHz | 0.4 | 18.9 [11.2–26.9] | incoherent |

would require a non-coherent modulation (energy-based MFSK or OOK, as ggwave uses) to tolerate the micro-speaker's phase incoherence, at much lower rates.

# 9 Summary of Findings

1. A laptop and a phone at contact range can exchange tens of kbps acoustically: **36,571 bps** Mac→Pixel 7a and **27,307 bps** Pixel→Mac, byte-verified, with all overhead counted — roughly two orders of magnitude above deployed data-over-sound systems, and 4.5–9× the project's own design document projection. The headline enabler was abandoning the inaudibility constraint, which widened the channel from 5 kHz to 16–22 kHz.

2. The measured channel, not the modeled one, dictated every major design parameter: CP length (delay spread is ∼10× the modeled value), pilot structure (clock skew, not Doppler, is the always-present phase enemy), speaker selection, and band edges (Pixel→Mac dies above 17 kHz).

3. Four stacked, simulation-invisible defects each individually killed the link; cheap discriminating experiments — including negative results that exonerated nonlinearity and the playback path — isolated them. Per-symbol pilot-EVM LLR weighting (soft erasures) was the highest-leverage single fix.

4. Verification discipline matters: a deterministic cross-language PRBS lets the receiving phone prove goodput on-device, and a conservative goodput definition (verified bytes over total airtime) keeps the result honest. The predecessor stack's 80× claim-to-measurement gap is the cautionary baseline.

5. Consumer audio platforms are an active part of the channel: Android silently zeroes capture based on UI visibility, processed capture paths corrupt QAM, and macOS fades stream tails. Budget for the OS.

# 10 Reproducibility

**Hardware.** MacBook Pro M4 (`Mac16,5`), macOS 26.5 (build 25F71); Google Pixel 7a, Android 16; iPhone 17 Pro Max (`iPhone18,2`). Signing team Primatech Paper Co LLC (V83C69HYSQ).

**Modem configuration.** 48 kHz PCM16; NFFT 2048; CP 768; comb pilots on every 8th used bin; 16-QAM; $K$=7 (171,133) convolutional code punctured to rate 3/4; frame-wide interleave;

CRC32 per 256-byte block; chirp preamble plus 2 sync symbols. Volume settings: Mac output 100%, Mac input ∼22/100, phone media at maximum.

**Repro commands.** `scratch/hw20k/ota_test.py` (Pixel); `scratch/hw20k/ios_ota_test.py` and `ios_campaign.py` (iPhone); `baselines.py` (ggwave/minimodem); `ios_phase_coherence.py` (phase jitter). iOS HIL details in `docs/IOS_HIL.md`.

**Branch commits at time of measurement.** `ios-hil-bulk` `0f80d8e`, `ultrasonic-band` `7699a6e`, `whitepaper` `d45ccbc`.

**Artifact paths.** Under `scratch/hw20k/data/`: `ios_campaign.jsonl`, `baselines.json`, `channel.json`, and `ios_phase_coherence.json`.

## References


[1] Quiet Modem Project. "Quiet." https://quiet.github.io/ (accessed February 12, 2026).

[2] Quiet Modem Project. "How It Works." https://quiet.github.io/docs/org.quietmodem.Quiet/how/ (accessed February 12, 2026).

[3] "Ultrasonic Communication Using Consumer Hardware." https://www.researchgate.net/publication/320575022_Ultrasonic_Communication_Using_Consumer_Hardware (accessed February 12, 2026).

[4] Google for Developers. "Get Started — Nearby Messages API for Android." https://developers.google.com/nearby/messages/android/get-started (accessed February 12, 2026).

[5] "minimodem — general-purpose software audio FSK modem." Ubuntu manpage. https://manpages.ubuntu.com/manpages/xenial/man1/minimodem.1.html (accessed February 12, 2026).

[6] kamalmostafa/minimodem, GitHub issue #30 (on the absence of integrated FEC). https://github.com/kamalmostafa/minimodem/issues/30 (accessed February 12, 2026).

[7] G. Gerganov. "ggwave: Tiny data-over-sound library." https://github.com/ggerganov/ggwave (accessed February 12, 2026).

[8] A. Zarandy, I. Shumailov, R. Anderson. "BatNet: Data transmission between smartphones over ultrasound." arXiv:2008.00136, August 2020. https://arxiv.org/abs/2008.00136 (accessed June 9, 2026).

[9] InvenSense (TDK). "AN-000225: Ultrasonic range sensing enables measured social contact" (Chirp white paper). https://invensense.tdk.com/wp-content/uploads/2020/08/AN-000225-Chirp-Social-Distancing-White-Paper-v1.0_8_3_20-2.pdf (accessed February 12, 2026).

[10] LISNR. "How Does Data over Audio Transmission Work?" https://lisnr.com/resources/blog/data-over-audio-transmission/ (accessed February 12, 2026).

[11] "Timing and Frequency Synchronization Using CAZAC Sequences." MDPI. https://www.mdpi.com/1424-8220/23/6/3168 (accessed February 12, 2026).

[12] "Analysis of the Frequency Offset Effect on Zadoff–Chu Sequence Timing Performance." arXiv. https://arxiv.org/pdf/1406.3412 (accessed February 12, 2026).

[13] Semtech Corporation. "AN1200.22: LoRa Modulation Basics." Application note.

[14] "Carrier and Sampling Frequency Offset Estimation and Tracking in OFDM Systems." IEEE. https://ieeexplore.ieee.org/document/4669503 (accessed June 9, 2026).

[15] "Pilot based sampling frequency offset estimation and correction for an OFDM system." U.S. Patent 7,782,752 B2. https://patents.google.com/patent/US7782752B2/en (accessed June 9, 2026).

[16] "Adaptive estimation and compensation of clock drift in acoustic echo cancellers." U.S. Patent 7,120,259 B1. https://patents.google.com/patent/US7120259B1/en (accessed June 9, 2026).

[17] "Capacity of Burst Noise-Erasure Channels With and Without Feedback and Input Cost." arXiv:1705.01596. https://arxiv.org/pdf/1705.01596 (accessed June 9, 2026).

[18] "Orthogonal time frequency space modulation for underwater acoustic communication systems: a review." Edith Cowan University Research Online. https://ro.ecu.edu.au/cgi/viewcontent.cgi?article=8367&context=ecuworks2022-2026 (accessed June 9, 2026).

[19] "Orthogonal time-frequency space modulation for underwater mobile acoustic communications." J. Acoust. Soc. Am. 157(2):1378, 2025. https://pubs.aip.org/asa/jasa/article-pdf/157/2/1378/20406915/1378_1_10.0035938.pdf (accessed June 9, 2026).

[20] "PowerPhone: Unleashing the Acoustic Sensing Capability of Smartphones." PMC. https://pmc.ncbi.nlm.nih.gov/articles/PMC12765216/ (accessed June 9, 2026).

[21] "Short-Range Audio Channels Security: Survey of Mechanisms, Applications, and Research Challenges." CRI-Lab. https://cri-lab.net/wp-content/uploads/2020/01/Audio_Channels_Security_Survey.pdf (accessed June 9, 2026).

[22] D. Asonov, R. Agrawal. "Keyboard Acoustic Emanations." IEEE Symposium on Security and Privacy, 2004. https://web.eecs.umich.edu/~genkin/teaching/fall2018/EECS598-12_files/kbdacoustic.pdf (accessed June 9, 2026).

[23] "NUSGuard: Smart Device Anti-Eavesdropping Protection Based on Near-Ultrasonic Interference." New Jersey Institute of Technology. https://researchwith.njit.edu/en/publications/nusguard-smart-device-anti-eavesdropping-protection-based-on-near/ (accessed June 9, 2026).

[24] "Dynamic Ultrasonic Jamming via Time–Frequency Mosaic for Anti-Eavesdropping Systems." Electronics 14(15):2960, MDPI. https://www.mdpi.com/2079-9292/14/15/2960 (accessed June 9, 2026).

[25] "Real Time Sound Processing on Android." JTRES 2016. https://steveyko.github.io/assets/pdf/rtdroid-sound-jtres16.pdf (accessed June 9, 2026).

[26] Android Developers (NDK). "Neon support." https://developer.android.com/ndk/guides/cpu-arm-neon (accessed June 9, 2026).

[27] "KFR: Fast, modern C++ DSP framework (SSE, AVX, AVX-512, ARM NEON, RISC-V RVV)." https://github.com/kfrlib/kfr (accessed June 9, 2026).

[28] "VoiceProcessingIO Audio Unit adds an unexpected input stream to Built-in output device (macOS)." Stack Overflow. https://stackoverflow.com/questions/64966350/voiceprocessingio-audio-unit-adds-an-unexpected-input-stream-to-built-in-output (accessed February 12, 2026).

[29] Apple Developer Documentation. "setPrefersEchoCancelledInput(_:)." https://developer.apple.com/documentation/avfaudio/avaudiosession/setprefersechocancelledinput(_:) (accessed February 12, 2026).

[30] H. Choi. "Audio chirp processing problems on OS X." http://henryomd.blogspot.com/2017/03/audio-chirp-processing-problems-on-os-x.html (accessed February 12, 2026).

[31] Faber Acoustical. "Measured: iPhone 15 Pro Max microphone frequency response and directivity." https://blog.faberacoustical.com/wpblog/2024/ios/iphone/measured-iphone-15-pro-max-microphone-frequency-response-and-directivity/ (accessed February 12, 2026).

[32] Apple Support. "Play high sample rate audio on your Mac." https://support.apple.com/en-us/108326 (accessed February 12, 2026).

[33] "Cyrinx: A High-Fidelity Adaptive Ultrasonic Transport Protocol — Technical Product Requirements Document & Research Report," February 2026. Internal design document; contrasted with the as-built system in `docs/PRD_VS_AS_BUILT.md` of the Cyrinx repository.

[34] "Ultrasonic-Band (Inaudible) Bulk PHY — Measured Results," `docs/ULTRASONIC_BAND.md` of the Cyrinx repository, June 2026. Fresh as of 2026-06-09.

[35] Cyrinx repository research records: `docs/ACOUSTIC_BULK_PHY.md` (research writeup), `scratch/hw20k/NOTES.md` (lab notebook), `scratch/hw20k/` (modem, harness, and diagnostic scripts), and `Apps/HIL/android/.../BulkDemod.kt` (on-device Kotlin decoder). All fresh as of 2026-06-09.